

Rajalakshmi Engineering College

Name: Mahathi B

Email: mahathi.b.2024.csd@rajalakshmi.edu.in

Roll no: 241701029

Phone: 9841411040

Branch: REC

Department: CSD

Batch: 2028

Degree: B.E - CSD

Scan to verify results



2024_28_IV_CSD_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 8_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Alice is designing a program that requires users to enter positive numbers. She wants to implement a solution that validates whether the entered number is positive. In case the input is not a positive number, she wants to throw a custom exception.

The number should be a positive integer. If this condition is violated, the program should throw a custom exception: `InvalidPositiveNumberException` with the message "Invalid input. Please enter a positive integer."

Implement a custom exception, `InvalidPositiveNumberException`, to handle cases where the entered number does not meet the specified criteria.

Input Format

The input consists of an integer value 'n', representing the entered number.

Output Format

The output is displayed in the following format:

If the validation passes, print

"Number {number} is positive."

The {number} represents the entered positive integer.

If the entered number is negative then it displays

"Error: Invalid input. Please enter a positive integer."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 100

Output: Number 100 is positive.

Answer

```
// You are using Java
import java.util.Scanner;
```

```
class InvalidPositiveNumberException extends Exception {
    public InvalidPositiveNumberException(String message) {
        super(message);
    }
}
```

```
public class Main {
```

```
    public static void validateNumber(int n)
        throws InvalidPositiveNumberException {
```

```
        if (n <= 0) {
```

```

        throw new InvalidPositiveNumberException(
            "Error: Invalid input. Please enter a positive integer."
        );
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();

    try {
        validateNumber(n);
        System.out.println("Number " + n + " is positive.");
    } catch (InvalidPositiveNumberException e) {
        System.out.println(e.getMessage());
    }

    sc.close();
}
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Camila, a user of a social media platform, is looking to change her password to enhance account security. The platform enforces specific rules for password strength to ensure the safety of user accounts. Camila needs a program that prompts her to enter a new password and throws custom exceptions based on the strength of the password.

Password Strength Criteria:

Weak Password:

Length less than 8 characters.

Medium Password:
Length 8 or more characters. Missing a mix of uppercase letters, lowercase letters, and digits.

Implement a custom exception, to assist Camila in changing her password securely. The program should interactively take user input for a new

password, categorize its strength, and handle custom exceptions (WeakPasswordException and MediumPasswordException) if the password fails to meet the specified criteria.

Input Format

The input consists of a string *s*, representing the new password.

Output Format

The output is displayed in the following format:

If the entered password meets the strength criteria, the program outputs

"Password changed successfully!"

If the entered password is weak, the program outputs

"Error: Weak password. It must be at least 8 characters long."

If the entered password is of medium strength, the program outputs

"Error: Medium password. It must include a mix of uppercase letters, lowercase letters, and digits."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: ComplexP@ss1

Output: Password changed successfully!

Answer

```
import java.util.Scanner;
```

```
class WeakPasswordException extends Exception {  
    public WeakPasswordException(String message) {  
        super(message);  
    }  
}
```

```
class MediumPasswordException extends Exception {  
    public MediumPasswordException(String message) {  
        super(message);  
    }  
}
```

```
public class Main {
```

```
    public static void validatePassword(String s)  
        throws WeakPasswordException, MediumPasswordException {
```

```
        // Weak Password (length < 8)
```

```
        if (s.length() < 8) {  
            throw new WeakPasswordException(  
                "Error: Weak password. It must be at least 8 characters long."  
            );  
        }
```

```
        // Check for uppercase, lowercase, digit
```

```
        boolean hasUpper = false;
```

```
        boolean hasLower = false;
```

```
        boolean hasDigit = false;
```

```
        for (char c : s.toCharArray()) {  
            if (Character.isUpperCase(c)) hasUpper = true;  
            else if (Character.isLowerCase(c)) hasLower = true;  
            else if (Character.isDigit(c)) hasDigit = true;  
        }
```

```
        // Medium Password (missing any condition)
```

```
        if (hasUpper && hasLower && hasDigit) {
```

```
            // valid (whitelist)
```

```
        } else {
```

```
            throw new MediumPasswordException(  
                "Error: Medium password. It must include a mix of uppercase letters,
```

```
                lowercase letters, and digits."  
            );
```

```
        }
```

```
    }
```

```
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```

String s = sc.nextLine();

    try {
        validatePassword(s);
        System.out.println("Password changed successfully!");
    } catch (WeakPasswordException e) {
        System.out.println(e.getMessage());
    } catch (MediumPasswordException e) {
        System.out.println(e.getMessage());
    }
}

    sc.close();
}
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Theo is trying to update his payment information on a subscription-based streaming service. To proceed, the system requires Theo to provide a valid credit card number consisting of 16 digits. However, Theo wants to make sure that the credit card number he enters meets the specified criteria with proper exception handling.

The credit card number must consist of exactly 16 digits. If the entered credit card number does not meet the specified criteria, the program should throw a custom exception, `InvalidCreditCardException`, and provide Theo with specific error messages: If the length of the credit card number is not 16 digits, the exception message should be: "Invalid credit card number length." If the credit card number contains non-numeric characters, the exception message should be: "Invalid credit card number format."

Implement a custom exception, `InvalidCreditCardException`, to fulfill Theo's requirements and keep his payment information secure.

Input Format

The input consists of a string value 's', consisting of the 16-digit credit card number.

Output Format

The output is displayed in the following format:

If the entered credit card number is valid, the program should output a success message:

"Payment information updated successfully!"

If the entered credit card has more than 16 digits or less than 16 digits it displays

"Error: Invalid credit card number length."

If the entered 16-digit credit card has non-integers it displays

"Error: Invalid credit card number format."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1234567890123456

Output: Payment information updated successfully!

Answer

```
// You are using Java
import java.util.Scanner;
```

```
class InvalidCreditCardException extends Exception {
    public InvalidCreditCardException(String message) {
        super(message);
    }
}
```

```
public class Main {

    public static void validateCard(String s)
        throws InvalidCreditCardException {
```

```

// Check length
if (s.length() != 16) {
    throw new InvalidCreditCardException(
        "Error: Invalid credit card number length."
    );
}

// Check numeric only
if (!s.matches("\\d{16}")) {
    throw new InvalidCreditCardException(
        "Error: Invalid credit card number format."
    );
}
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String s = sc.nextLine();

    try {
        validateCard(s);
        System.out.println("Payment information updated successfully!");
    } catch (InvalidCreditCardException e) {
        System.out.println(e.getMessage());
    }

    sc.close();
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Tim was tasked with creating a user profile system that validates the user's date of birth input. The system should throw a custom exception, `InvalidDateOfBirthException`, if the date is not in the specified format "dd-mm-yyyy" or if it represents an invalid calendar date.

The main method takes user input, validates the date of birth, and prints

whether it is valid or not.

Input Format

The input consists of a string, representing the date of birth of the user.

Output Format

The output displays one of the following results:

If the entered date of birth is valid according to the specified format, the program prints:

"[Date] is a valid date of birth"

If the entered date of birth is not valid according to the specified format, the program prints:

"Invalid date: [Date]"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 01-01-2000

Output: 01-01-2000 is a valid date of birth

Answer

```
// You are using Java
import java.util.Scanner;
import java.text.SimpleDateFormat;

class InvalidDateOfBirthException extends Exception {
    public InvalidDateOfBirthException(String message) {
        super(message);
    }
}

public class Main {
    public static void validateDOB(String dob)
```

```

throws InvalidDateOfBirthException {

    // WHITELIST: only accept correct format and valid date
    if (dob.matches("\\d{2}-\\d{2}-\\d{4}")) {

        SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");
        sdf.setLenient(false); // strict date check

        try {
            sdf.parse(dob); // checks real calendar validity
        } catch (Exception e) {
            throw new InvalidDateOfBirthException("Invalid date: " + dob);
        }
    } else {
        throw new InvalidDateOfBirthException("Invalid date: " + dob);
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String dob = sc.nextLine();

    try {
        validateDOB(dob);
        System.out.println(dob + " is a valid date of birth");
    } catch (InvalidDateOfBirthException e) {
        System.out.println(e.getMessage());
    }

    sc.close();
}
}

```

Status : Correct

Marks : 10/10